NATIONAL ECONOMICS UNIVERSITY · BFT65B – BIG DATA ANALYTICS IN FINANCE

A Big-Data Pipeline for Bank-Failure Early Warning

Final examination – individual research report

Table 1. Key facts required on the first page.

Item	Value
Student	Nguyễn Trung Kiên — student ID 11230982
GitHub repository (public)	https://github.com/kientainang2005-arch/Final-exam-Bigdata
Download method and date	FDIC BankFind public API (https://api.fdic.gov/banks), own resumable script src/01_download.py adapted from the course starter; downloaded on 6 November 2026
Row counts	Financials 1,682,616 rows (170 quarters 1984Q1–2026Q2, 24,064 banks; equal to the API total in every quarter) · Failures 4,117 · Institutions 27,835
Clean panel	1,681,405 bank-quarters; 1,663,876 labelled rows up to 2025Q2 with 10,860 positives (0.65%)
Development period	1986Q1–2006Q4: 1,049,999 bank-quarters, 7,689 positive rows, 2,043 failing banks
Gap (unused)	2007Q1–2007Q4 (4 quarters, 34,754 rows)
Test period (used once)	2008Q1–2025Q2: 435,446 bank-quarters, 2,134 positive rows, 545 failing banks

Abstract

This report builds a reproducible early-warning data pipeline on four decades of US Call Reports. The data are downloaded with a resumable, self-verifying procedure and stored as a typed, zstd-compressed Parquet layer partitioned by year, which is 60% smaller than CSV and reads a three-year slice 472 times faster. An automated audit finds and handles 18 classes of data problems, among them year-to-date income flows, zeros that mean ‘not reported’, schema drift, failures that cannot be identified from structure codes and static attributes that leak the future. The core transformations run in Polars' lazy engine in 1.0 s, 8× faster than an identical pandas pipeline with 39% less memory; PySpark reproduces them to within 6e-14. Forty point-in-time features pass an automatic truncation test (identical values when future data are removed). A gradient-boosting model selected by walk-forward validation on 1986Q1–2006Q4 reaches a test ROC-AUC of 0.983 and PR-AUC of 0.554 on 2008Q1–2025Q2; at the cost-based threshold it flags 526 of 545 failing banks, with a median lead of 4 quarters.

1. Introduction

The supervision department plans an early-warning system (EWS) that ranks banks by failure risk. Its value depends first on the data pipeline: if the data are incomplete, mis-typed or contain information from the future, no model on top of them can be trusted. Because Vietnamese supervisory data are not public, the pipeline is built and tested on the US FDIC data, which cover three crises (1984–1993, 2008–2014, 2023). The report answers three questions:

- **Q1** – How should four decades of bank reports be stored and processed on a laptop, and when would a Spark cluster be needed? (Sections 2 and 4)
- **Q2** – What quality problems do the raw data contain, and how does the pipeline handle them without look-ahead bias? (Sections 3 and 5)
- **Q3** – Can the pipeline's output support an honestly validated early-warning model? (Section 6)

Every claim is backed by a measurement produced by the code in the repository; all tables and figures are written to results/ by the pipeline and this report is generated from those files (report/build_report.py). The machine is a 2024 laptop (Snapdragon X, 8 cores, 16 GB RAM, Windows 11; Python runs as x64 under emulation), so absolute timings are conservative.

2. Acquisition and storage (Part 1)

2.1 Acquisition

src/01_download.py (adapted from the course starter) pages through the /financials endpoint one quarter at a time (10,000 rows per call, exponential back-off on HTTP or network errors) and writes each quarter atomically (temporary file, then rename), so an interrupted run never leaves a half-written file. A JSON manifest records, per quarter, the rows received, the API's own total, a timestamp and a completeness flag; re-running the script skips quarters that are already complete, which makes the download **resumable**. Failures and Institutions are small and are fetched in full. The full download took about 25 minutes.

Completeness check. For all 170 quarters the number of rows received equals the API total; the sum is 1,682,616 rows, there are 0 duplicate (CERT, REPDTE) keys and 0 incomplete quarters (Table 11 in the appendix and results/dq_rows_per_quarter.csv). The number of reporting banks falls from about 18,000 in 1985 to about 4,300 in 2026 (Figure A1 in the appendix), the expected consolidation trend; no quarter shows a gap or spike that would indicate a partial download.

Field choice. The starter's 30 Call Report fields were extended to 45 (plus CERT, REPDTE and NAME), chosen from the FDIC dictionary to cover every CAMELS dimension: capital (EQ, leverage and risk-based ratios), asset quality (non-current, past-due and non-accrual assets, ORE, charge-offs, provisions), earnings (net income, interest income and expense, non-interest items, dividends), liquidity and funding (cash, securities, core and brokered deposits, volatile liabilities, FHLB borrowings) and loan concentration (construction, multifamily, non-farm non-residential, 1–4 family, C&I, consumer, agricultural loans). Commercial real-estate concentration was the main driver of the 2008–2012 failures, so the real-estate breakdown is essential. BKCLASS, SPECGRP and STNAME are taken from Financials because there they are recorded per quarter (point-in-time), unlike in Institutions.

2.2 Storage layout

The pipeline keeps three layers: a raw landing zone (one Parquet file per quarter, exactly as received), a curated layer with one explicit schema, and derived tables (panel, features) that can always be rebuilt.

- **Format:** Parquet (columnar). Credit-risk work reads a few columns of many rows; a columnar file lets the engine read only those columns and skip row groups using min/max statistics.
- **Schema:** CERT Int32, REPDTE Date, amounts Int64 (whole thousands of USD), ratios Float64, state and charter class Categorical. The raw files do not agree on types: 20 columns change between int64 and double across quarters (Section 3), which breaks naive multi-file reads; the curated layer casts once and for all.
- **Partitioning:** by year (Hive style, year=YYYY/), 43 files of 1.2–7.8 MB (2026 holds only two quarters). Analyses are mostly time-sliced (crisis windows, train/test periods), so a year filter becomes partition

pruning. Partitioning by quarter would create 170 small files, and by CERT more than 24,000 tiny files – the classic small-file problem.

- **Ordering and compression:** rows sorted by (CERT, REPDTE) inside each file so that window operations per bank read contiguous data and run-length/dictionary encoding works well; zstd level 3 compression.

Table 2. Storage benchmark on the full panel (1.68 M rows × 49 columns). One warm-up then median of 5 runs; Polars reader unless stated.

Layout	Size (MB)	Write (s)	Read all (s)	Read 5 cols (s)	Filter 2008–10, 5 cols (s)	Size / CSV
CSV (uncompressed)	518.9	0.593	7.537	6.749	6.138	1.00
CSV + gzip	213.6	40.087	5.559	2.304	2.420	0.41
Parquet snappy (1 file)	241.7	0.380	0.110	0.010	0.011	0.47
Parquet zstd-3 (1 file)	195.1	0.615	0.142	0.014	0.012	0.38
Parquet zstd-3, partitioned by year	207.6	0.889	0.187	0.028	0.013	0.40
CSV read with pandas (baseline)	518.9	–	5.477	1.363	–	1.00

Parquet with zstd is 62% smaller than CSV and the partitioned layout 60% smaller. Reading everything is 40× faster, because CSV must be parsed and its types inferred, whereas Parquet is already typed and compressed in column chunks. The gain is largest when the query needs only a few columns (241× faster) or a time slice (472× faster), the typical supervisory access pattern: CSV has no column projection or statistics, so every query scans all 519 MB. gzip makes CSV small but is the slowest to write and still has to be fully decompressed and parsed. The partitioned layout costs slightly more space than a single zstd file (per-file overhead) and is slightly slower for full scans, but it is the fastest for time filters and allows new quarters to be appended by rewriting one small partition instead of the whole file – the decisive operational advantage for a quarterly-updated EWS.

3. Data quality and the bank-quarter panel (Part 2)

3.1 Automated audit

src/03_quality.py runs a fixed battery of 18 checks on every run and writes results/dq_report.md with the evidence tables (the full summary is Table 10 in the appendix). The most consequential findings are:

- **Year-to-date income flows.** All income-statement items accumulate from January: median INTINC/ASSET is 1.7% in Q1, 3.5% in Q2, 5.2% in Q3 and 6.8% in Q4, while the FDIC's ROA is flat across quarters because it is already annualised. Using NETINC/ASSET directly would inject a seasonal saw-tooth. The pipeline de-cumulates: flow_q = YTD_q − YTD_{q−1} when the previous quarter of the same year exists, Q1 = YTD, otherwise null.
- **Zeros that mean 'not reported'.** Risk-based capital ratios are exactly 0 in 100% of 1984–89 rows (they did not exist before Basel I) and in 37% of 2020–26 rows (small banks that opted into the Community Bank Leverage Ratio stop reporting them). A model would read 0% capital as insolvency. The ratios are therefore not used; equity/assets, which exists in all eras, measures capital instead.
- **Structural missingness.** Some items were introduced late: 30–89-day past-due assets are missing for 100% of rows before 2000, FHLB borrowings for 82–87% before 2000 (Figure A2). They are kept with a missing-indicator and a training-period median, never imputed from later data.
- **Impossible values.** 860 negative values in stock amounts that cannot be negative (mostly ORE) are set to null; 15,659 rows with negative equity are kept because they are genuine and highly informative; 18,077 risk-based ratios above 100% come from banks with almost no risk-weighted assets. Ratios are recomputed from amounts with safe division and winsorised with training-period quantiles.
- **Schema drift.** 20 columns change physical type across raw files (int64 vs double), fixed by the curated schema.
- **Failure identification.** The Failures table holds 2,848 FAILURE and 411 ASSISTANCE events in scope. Its 488 rows without CERT are all pre-1984. 44 CERTs appear more than once (37 assisted and later failed). In 271 failures the institution kept filing after FAILDATE – 226 of them savings institutions placed in RTC conservatorship in 1989, where FAILDATE is the takeover date.

- **Structure codes are not a failure flag.** Code 211 is 99.3% failures, but only 83.6% of failed banks carry it; the others carry merger-type codes (213, 217, 235) that also occur for healthy mergers. Using CHANGEC would mislabel about one failure in six as a merger.
- **Point-in-time traps.** 4,518 banks (18.8%) changed charter class over time and 249 changed state, but Institutions stores only the latest values; ACTIVE, ENDEFYMD and CHANGEC describe the end of a bank's life. Joining them to old quarters would leak the future, so Institutions is used only to analyse exits, never for features.

3.2 Panel construction and exits

src/04_panel.py (logic in src/transforms.py) builds the panel in one lazy Polars query: drop rows with zero assets (1,211 rows that cannot form ratios), null impossible negatives, add a quarter index qi = 4·year + quarter − 1, de-cumulate the flows, attach the first FAILURE date per CERT and derive the label and exit type.

- **Label:** y(i,q) = 1 if bank i fails in (q, q+4]: fail_qi > qi and fail_qi ≤ qi + 4, where fail_qi is the quarter containing FAILDATE. Rows dated on or after FAILDATE are removed (the event is public by then).
- **Failure vs. other exits:** failure if and only if the CERT has a RESTYPE = FAILURE event. Open-bank assistance is not a failure: the charter survives and 176 of the 411 assisted banks keep reporting afterwards. All other exits (mergers, voluntary liquidations, charter conversions) keep label 0 in all their rows and simply leave the panel after their last report; they are not dropped, which would create survivorship bias, and not labelled as failures, which would teach the model that healthy acquisition targets are risky.
- **Label completeness:** for modelling only quarters up to 2025Q2 are used, so that every label window (q, q+4] is observed.

The panel has 1,681,405 bank-quarters for 24,062 banks: 2,839 failed, 16,910 exited otherwise and 4,313 still report in 2026Q2. 2837 of the 2845 in-scope failed banks have at least one positive row; the remainder stopped reporting more than four quarters before failing. The positive rate is 0.65%, i.e. about 1 in 150 bank-quarters (Figure 1).

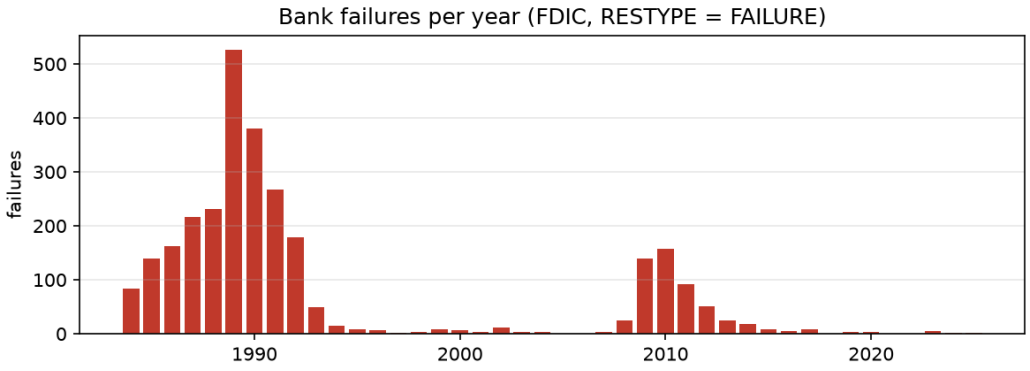


Figure 1. Bank failures per year (RESTYPE = FAILURE). Failures cluster in 1984–1993 and 2008–2014.

4. Scalable processing (Part 3)

4.1 Polars implementation and how the engine executes it

All transformations are written once as Polars LazyFrame expressions (src/transforms.py, src/features.py) and reused by the panel, the feature table, the benchmark and the truncation test. Nothing is computed until collect(); Polars first builds a logical plan and optimises it (the plans are saved in results/polars_plan_panel_*.txt):

- **Projection pushdown:** only the columns used downstream are read from Parquet. In the panel plan (appendix E) the failures scan reads PROJECT 3/16 COLUMNS and the financials scan drops the unused partition column; a query that selects five columns never decodes the other 44 (‘Read 5 cols’ in Table 2).
- **Predicate pushdown:** the filter ASSET > 0 written after the scan appears inside it as SELECTION: col("ASSET") > 0, and the RESTYPE = FAILURE filter is pushed into the failures scan; with a REPDTE or year filter, Hive partitions outside the range are never opened.

- **Common-subexpression elimination:** the optimised plan contains cached sub-expressions (__POLARS_CSER_…) – e.g. shift(1).over(CERT) of the quarter index is computed once and reused for all eight de-cumulated flows.
- **Parallel window and join execution:** .over('CERT') is evaluated as a parallel group-by on all cores; lags are self-joins on (CERT, qi−4), which are hash joins rather than Python loops. Rolling statistics use rolling_*_by on the quarter index, so a missing quarter shrinks the window instead of silently using an older report.
- **Streaming engine:** collect(engine='streaming') processes the data in batches, trading some speed for a lower memory peak.

4.2 Benchmark against pandas

The same pipeline (clean → de-cumulate → label → ratios → 4-quarter lags → trailing rolling statistics → state medians) was written idiomatically in pandas (vectorised, groupby/shift/merge/rolling, no Python loops) and run against Polars in three modes. To be fair, every run starts in a fresh subprocess (cold Python, same files), peak memory is the process peak working set, and each configuration is run three times (median reported). The outputs are compared value by value.

Table 3. Same pipeline on 1,681,405 output rows. Max |pandas – Polars| over 15 output columns: 4e-09; null patterns identical.

Engine	Time, median (s)	Time, min (s)	Peak memory (MB)	Speed-up
pandas	7.86	7.85	2,569	1.0×
polars-eager	1.36	1.35	4,450	5.8×
polars-lazy	1.04	1.00	1,561	7.6×
polars-streaming	1.37	1.35	1,104	5.7×

Polars lazy is 7.6× faster than pandas and uses 39% less memory. The eager variant shows where the gain comes from. With the same expressions but every intermediate table materialised, Polars eager is still 5.8× faster than pandas – the vectorised, multi-threaded Rust kernels – but it needs the most memory of all engines (4,450 MB, 2.9× the lazy run) and is 31% slower than lazy. The lazy query optimiser adds the rest: it fuses steps, prunes columns and never allocates the intermediate tables, which matters more for memory than for time. Streaming lowers the memory peak further to 1,104 MB at a small cost in time, because sort-based window operations limit how much of this pipeline can be streamed.

Scaling. To see how the engines behave beyond the real data size, the panel was replicated 2, 4 and 8 times with disjoint CERTs (Figure 2).

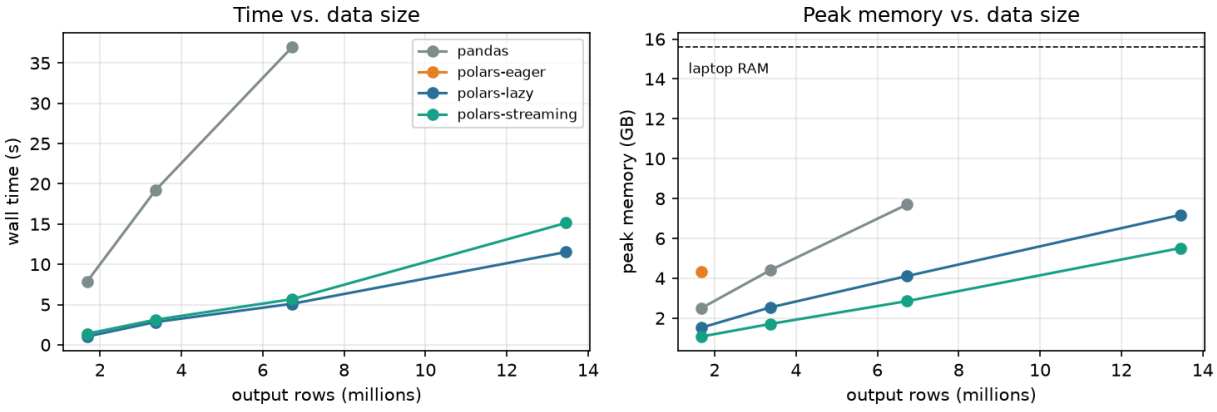


Figure 2. Wall time and peak memory vs. data size (×1, ×2, ×4, ×8 replicas of the panel).

pandas grows to 37 s and 7.7 GB at ×4 (6.7 M rows) – half of the laptop's RAM – and was not run at ×8. Polars processes ×8 (13.5 M rows) in 12–15 s with 5.5 GB (streaming) to 7.2 GB (lazy). Time and memory grow roughly linearly.

4.3 Reproduction in PySpark

notebooks/08_spark_demo.ipynb re-implements the key steps in the Spark DataFrame API (local[*], AQE on, 16 shuffle partitions): cleaning, de-cumulation with lag() over a window partitioned by CERT, the broadcast join

with failures, the label, eight ratio features, the 4-quarter self-join, gap-safe rolling windows (rangeBetween(−7, 0) on the quarter index) and state medians. The result is compared with Polars row by row:

Table 4. Spark vs. Polars on the same 1.68 M rows (sorted by CERT, quarter).

Column	Null mismatches	Max |diff|	Sum (Polars)	Sum (Spark)
y	0	0.0e+00	10871.000	10871.000
eq_ta	0	0.0e+00	179639.553	179639.553
npa_ta	0	0.0e+00	26116.076	26116.076
npl_ln	0	0.0e+00	30999.040	30999.040
roa_q	0	0.0e+00	44057.061	44057.061
nim_q	0	0.0e+00	69448.775	69448.775
texas	0	0.0e+00	310183.344	310183.344
cre_ta	0	0.0e+00	254040.934	254040.934
brokered_dep	0	0.0e+00	28849.298	28849.298
d4_eq_ta	0	0.0e+00	-3629.724	-3629.724
d4_npa_ta	0	0.0e+00	1157.245	1157.245
g4_asset	0	0.0e+00	223481.676	223481.676
roa_vol8	0	5.7e-14	29539.666	29539.666
roa_4q	0	1.8e-15	20543.047	20543.047
state_npa_med	0	0.0e+00	16303.038	16303.038

All rows and columns match; the only differences (≤ 6e-14) are floating-point summation order in the rolling standard deviation. Spark needed 105 s for this job against 2.1 s for Polars on the same laptop.

Reading the physical plan (explain('formatted') in the notebook, read bottom-up). (1) Scan parquet: ReadSchema lists only the 17 of 49 columns the job needs and PushedFilters contains GreaterThan(ASSET, 0) – the same projection and predicate pushdown as Polars; with a year filter (notebook section 1) the scan also shows PartitionFilters, i.e. partition pruning. (2) Exchange hashpartitioning(CERT, 16) → Sort → Window: a shuffle that sends each bank's history to one partition so lag() can de-cumulate the flows. (3) The failures table is aggregated (HashAggregate – Exchange – HashAggregate), shipped to every task by BroadcastExchange and joined with BroadcastHashJoin LeftOuter, so the 1.68 M-row panel is not shuffled for this join. (4) The 4-quarter self-join is a SortMergeJoin LeftOuter with both sides re-partitioned on hashpartitioning(CERT, qi). Its right-hand side scans the Parquet files a second time and contains no Window: the optimiser saw that eq_ta, npa_ta and ASSET do not depend on the de-cumulated flows and pruned that work, but it still reads the data twice. (5) A new Exchange on CERT for the rolling windows – the (CERT, qi) partitioning of the join does not satisfy a window partitioned by CERT – and (6) an Exchange on (STNAME, qi) for the state median. Five shuffles in total: on a cluster each moves data across the network, on one machine it is pure serialisation overhead, which is why Spark is slower than Polars here. Spark is also lazy: without cache() every action re-executes the whole lineage from the scan, so the MLlib part caches the feature table first (35 s once, instead of once per pass of every estimator).

4.4 When would a cluster be justified?

The full 42-year panel occupies 208 MB on disk and at most 1.5 GB in memory during the whole transformation pipeline, which takes 1.0 s. From the scaling experiment the lazy engine needs roughly 573 bytes per output row, so with 70% of 16 GB usable a single laptop can process about 20 M bank-quarter rows in memory – about 12 times the FDIC panel – and the streaming engine and out-of-core sinks push this further. On this data Spark is 49× slower because of JVM start-up, task scheduling and shuffle overhead. A cluster is justified only when one of these conditions holds:

- the working set exceeds one machine's memory even with streaming – e.g. loan-level credit-registry data (hundreds of millions of loan-months) or transaction data, not quarterly bank aggregates;
- the job must finish within a fixed window that one machine cannot meet (e.g. daily stress tests over many scenarios);

- data arrive continuously from many sources and must be processed with fault tolerance (Spark retries failed tasks), or many users and jobs share the same data lake.

For a quarterly bank-level EWS – the FDIC case and even more so Vietnam's roughly 50 banks – none of these hold: Polars on one machine is the right tool, and Spark should be kept as a demonstrated, tested fall-back for a future move to loan-level data.

5. Feature pipeline (Part 4)

Features follow the CAMELS framework used by supervisors, plus dynamics and context. Table 5 groups them; the full dictionary with rationale, missing share and medians for failing and other banks is Table 12 in the appendix.

Table 5. Feature groups (40 features).

Group	Features
Capital	eq_ta
Asset quality	npl_ln, npa_ta, pd30_ta, ore_ta, texas, alll_ln, nco_ln, prov_ta
Management / efficiency	efficiency
Earnings	roa_q, nim_q, div_payer
Liquidity / funding	liq_ta, loans_ta, coredep_ta, brokered_dep, volliab_ta, fhlb_ta
Sensitivity / concentration	cof_ta, cre_ta, constr_ta, resid_ln, ci_ln, cons_ln, ag_ln
Size	log_assets
Dynamics (change, growth, volatility, history)	d4_eq_ta, d4_npa_ta, d4_npl_ln, d4_roa_q, g4_asset, g4_lnlsgr, roa_vol8, roa_4q, n_reports
Context (state and national stress)	state_npa_med, state_roa_med, npa_vs_state, fails_trailing4q_us

Examples of the separation in the data (medians, failing vs. other bank-quarters): equity/assets 1.7% vs 9.2%, Texas ratio 1.67 vs 0.07, non-performing assets 10.7% vs 0.7% of assets, quarterly ROA -2.9% vs 1.0%, 4-quarter asset growth -8% vs 5%. Rapid deterioration (4-quarter changes) and stress in the bank's state add information that a single snapshot misses.

Point-in-time contract. A feature for bank i in quarter q may use (a) bank i's reports dated ≤ q, (b) aggregates of other banks' reports dated ≤ q (state medians), and (c) failures with FAILDATE ≤ q (public). Lags are self-joins on (CERT, qi−4); trailing windows are defined on the quarter index; no statistic of the whole sample is computed in the feature code. Static Institutions attributes and exit fields are never used.

Automatic leakage test. src/06_features.py recomputes all 40 features from data truncated at a cut-off T (Financials with REPDTE ≤ T, Failures with FAILDATE ≤ T) and compares them with the features from the full data for every row dated ≤ T. For the three cut-offs 1991-12-31, 2009-06-30, 2023-03-31 (540,983, 1,279,056, 1,622,389 rows) the values and null patterns are identical (max |diff| = 0); the script stops with an error otherwise. This turns ‘no look-ahead’ from a claim into a tested property.

Preprocessing fitted on training data only. All fitted steps live in an sklearn Pipeline that is re-fitted inside every training block: DropEmpty (drops columns that are entirely missing in the training block, e.g. past-due assets before 2000), Winsorizer (clips each feature to its 0.5%/99.5% training quantiles), SimpleImputer (training median, with missing-indicator columns), StandardScaler (logistic models only). Validation and test rows are only transformed.

6. Modelling and validation (Part 5)

6.1 Split and validation scheme

Development 1986Q1–2006Q4, **gap** 2007Q1–Q4, **test** 2008Q1–2025Q2. 1984–85 are used only as history for the lags. The gap of four quarters is necessary because the label of a development row in 2006Q4 looks ahead to 2007Q4; without it, development labels would overlap the test period. The choice is deliberate: the model learns from the savings-and-loan and early-1990s crises and must then recognise a different crisis (2008–2012, driven by real-estate lending) and the 2023 failures – a genuine out-of-time and out-of-crisis test with 545 failing banks,

far above the minimum of 50. A test period starting later (e.g. 2012) would leave too few failures for stable estimates.

Inside the development period, model selection uses **expanding-window walk-forward validation** (Table 6): each model is trained on all quarters up to a cut-off and validated on a later block, with four unused quarters in between for the same reason as the main gap. Random k-fold would mix 1990 and 1988 rows of the same bank and crisis and overstate performance.

Table 6. Walk-forward folds inside the development period (chosen model).

Fold (expanding window, 4-quarter gap before validation)	Val. rows	Val. positives	PR-AUC (HGB)	ROC-AUC (HGB)
train ≤ 1987-12 | val 1989-03..1990-12	127,191	2,736	0.458	0.948
train ≤ 1989-12 | val 1991-03..1992-12	116,928	1,157	0.516	0.977
train ≤ 1991-12 | val 1993-03..1995-12	155,446	153	0.378	0.992
train ≤ 1994-12 | val 1996-03..2006-12	440,240	168	0.136	0.920

Metrics. With 0.65% positives, accuracy is meaningless (predicting 'no failure' everywhere scores 99.5% on the test period). The primary metric is PR-AUC (average precision), which focuses on the rare class; ROC-AUC, recall and precision among the top 1% and 5% of bank-quarters (what a supervisor with limited examination capacity would review) and the Brier score (calibration) complete the picture.

6.2 Models and development results

Table 7. Walk-forward results on the development period (mean over the 4 folds).

Model	PR-AUC	ROC-AUC	Recall top 1%	Recall top 5%	Brier
Baseline: logistic, 5 CAMELS ratios	0.163	0.903	0.493	0.743	0.008
Logistic, all 40 features (L2)	0.242	0.901	0.518	0.741	0.007
Histogram gradient boosting (HGB)	**0.372**	**0.959**	**0.573**	**0.823**	**0.006**

The baseline is the classic supervisory logit on five CAMELS ratios (equity/assets, NPA/assets, ROA, liquid assets, log size). The richer logit uses all 40 features with L2 regularisation. HGB (scikit-learn's histogram gradient boosting) captures non-linear thresholds and interactions such as 'high CRE concentration and falling capital' and handles missing values natively; its hyper-parameters were chosen from 8 combinations by mean walk-forward PR-AUC (learning rate 0.05, 15 leaves, at least 200 rows per leaf, 300 trees). HGB is clearly best on every development metric and was selected; the model, features and threshold were frozen to results/frozen_choices.json before the test period was read.

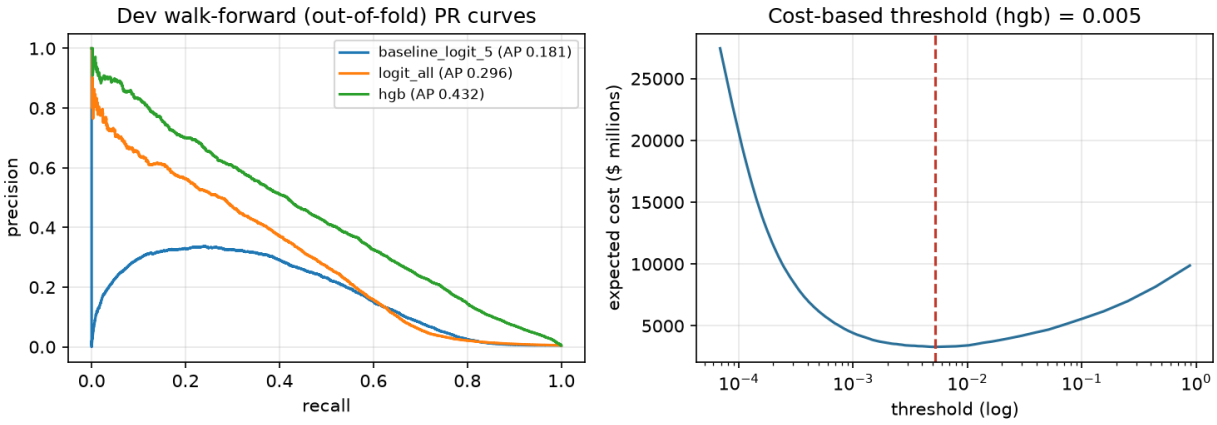


Figure 3. Left: pooled out-of-fold PR curves on the development period (AP of the pooled predictions, therefore different from the per-fold means in Table 7). Right: expected cost vs. alarm threshold.

6.3 Alarm threshold from costs

The threshold minimises expected cost C_FN·FN + C_FP·FP on the pooled out-of-fold development predictions. Both costs come from development-period data: the median loss to the deposit insurance fund of the 2,034 failures in 1986–2006 with a recorded COST is $9.54 M (median loss rate 20.2% of the failed bank's assets).

Assuming that early supervisory action avoids 25% of that loss gives a missed-failure cost C_FN = $2.39 M. A false alarm triggers an unnecessary targeted examination, valued at 0.1% of the median bank's assets: C_FP = $66 k. The ratio is 36:1 and the cost-minimising threshold is p = 0.0053. Because the 25% and 0.1% are assumptions, Table 8 shows how the threshold responds to C_FP.

Table 8. Sensitivity of the threshold to the false-alarm cost (development out-of-fold predictions).

C_FP × (1 = base case)	Threshold	TP	FP	Recall	Precision
0.25	0.0009	3,838	55,168	91.1%	6.5%
0.5	0.0016	3,676	37,443	87.2%	8.9%
1.0	**0.0053**	**3,326**	**17,802**	**78.9%**	**15.7%**
2.0	0.0103	3,105	11,710	73.7%	21.0%
4.0	0.0239	2,722	6,832	64.6%	28.5%

6.4 Single evaluation on the test period

Table 9. Test period 2008Q1–2025Q2 (435,446 bank-quarters, 2,134 positives). Each model at its own development-chosen cost threshold. Evaluated once.

Model	PR-AUC	ROC-AUC	Recall top 1%	Recall @thr	Precision @thr	FP	FN	Cost ($ bn)
Baseline: logistic, 5 CAMELS ratios	0.542	0.978	73.1%	78.6%	29.4%	4,021	457	1.36
Logistic, all 40 features (L2)	0.558	0.980	74.2%	74.9%	35.7%	2,880	535	1.47
Histogram gradient boosting (HGB)	**0.554**	**0.983**	**75.9%**	**88.4%**	**19.7%**	**7,697**	**247**	**1.10**

The selected HGB model reaches ROC-AUC 0.983 and PR-AUC 0.554. At the frozen threshold it catches 88.4% of positive bank-quarters with 19.7% precision and has the lowest expected cost of the three models. From the bank perspective, 526 of the 545 banks that failed in the test period (96.5%) were flagged at least once before failing, with a median lead time of 4 quarters (Figure 4). Its accuracy, 98.2%, is below the 99.5% of the trivial 'no failure' rule – an illustration of why accuracy was not used.

Two results deserve an honest reading. First, all models score much higher on the test period than in development; the 2008–2012 failures were preceded by large, visible deteriorations in capital and asset quality, which makes them easier to rank than the heterogeneous 1990s failures in the later development folds. Second, the gain of HGB over the simple logit, large in development, is small on the test period (the 40-feature logit even has a slightly higher PR-AUC). HGB's advantage on the test set lies in recall at the chosen operating point – it misses 247 positive bank-quarters against 457 for the baseline – at the price of more false alarms, which the cost function accepts because a miss is 36 times as expensive. Alarms track the crisis closely and fall to a few dozen per quarter after 2014.

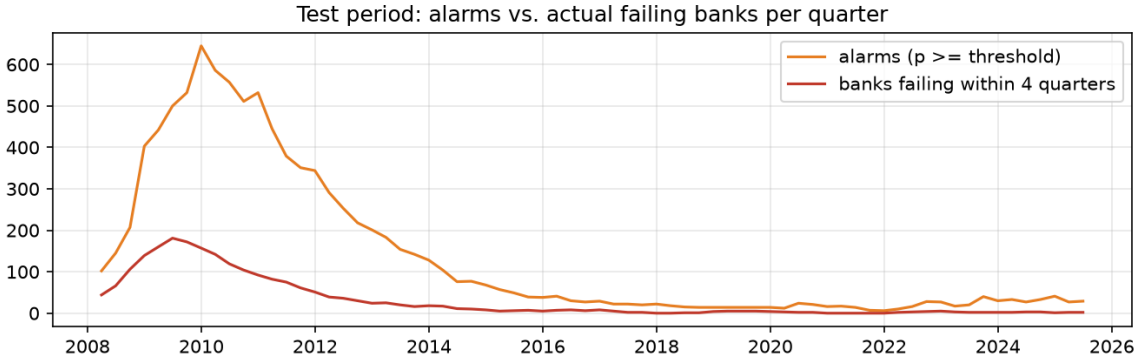


Figure 4. Test period: number of alarms vs. banks failing within four quarters, by quarter.

Spark MLlib demo. The notebook fits an MLlib pipeline (Imputer → VectorAssembler → StandardScaler → LogisticRegression, 14 features, quantile clipping from the training block) on 1986–1994 and validates on 1996–2006, the last development fold – the test period is not touched. It reaches PR-AUC 0.059 and ROC-AUC 0.865

(fit 56 s). Its ROC-AUC is close to the scikit-learn logits on the same fold (baseline 0.877, all features 0.888), but its PR-AUC is lower (sklearn: 0.099 and 0.133): the demo uses only 14 features, no missing-value indicators and a different regularisation, and with 168 positives among 440,000 rows PR-AUC is very sensitive to the few top-ranked cases. The point of the demo is that the whole modelling stage – imputation, scaling, fitting, evaluation – runs as a distributed Spark ML pipeline on the same data, so it can be moved to a cluster if the data ever outgrow one machine.

7. Conclusions (Part 6)

Q1 – Storage and processing. Store the reports as a typed, zstd-compressed Parquet layer partitioned by year (208 MB instead of 519 MB of CSV, 472× faster time-sliced reads) and process it with Polars' lazy engine (1.0 s and 1.5 GB for the whole pipeline, 8× faster than pandas). Spark gives identical results but is 49× slower on one machine; a cluster becomes necessary only beyond roughly 20 M rows per job, for loan- or transaction-level data, strict processing windows or many concurrent users.

Q2 – Data quality without look-ahead. The raw data contain year-to-date flows, zeros that mean 'not reported', fields introduced decades apart, impossible values, type drift, multiple and non-standard failure events and static attributes that describe the end of a bank's life. Each is detected automatically and handled with information available at the time: de-cumulation within the year, null instead of zero, missing indicators and training-period statistics, failures defined only by the Failures table, quarter-specific attributes. The truncation test proves that no feature changes when future data are removed.

Q3 – Honest validation. Yes. With a time-ordered split, a four-quarter gap, walk-forward selection, preprocessing fitted inside each training block, a cost-based threshold frozen before testing and a single test evaluation, the pipeline's output supports a model that ranks banks well out of sample (test ROC-AUC 0.983) and warns about 97% of failures about a year ahead.

Limitations

- The API serves the latest amended Call Reports, not the vintages available at the time, and reports are published 30–60 days after quarter end; a production EWS would need vintage data and a publication lag in the label window.
- Failures cluster in two crises; the test period is dominated by 2009–2011, so performance in calm periods (2015–2025, a handful of failures per year) is estimated with large uncertainty.
- The two cost parameters (avoidable share of loss and examination cost) are assumptions; Table 8 shows the threshold is sensitive to them. Assisted banks are treated as non-failures, a defensible but debatable choice.
- Timings come from one laptop running x64 Python under emulation on an ARM CPU; ratios between engines are more reliable than absolute numbers. The ×2–×8 scaling uses replicated data with the same distribution.

What would change for Vietnam

- **Scale and tools:** about 50 credit institutions reporting monthly to the SBV is a much smaller panel, so the same Parquet + Polars design is more than enough; Spark becomes relevant only for loan-level data from the Credit Information Centre (CIC).
- **Labels:** Vietnam has had very few formal bank failures; distress appears as special control, compulsory acquisition at zero price (2015) or restructuring under Law 32/2024. The label must be redefined (e.g. placement under special control or a large capital shortfall) and will be even rarer – US data or a transfer-learning approach could provide the prior.
- **Data definitions:** Vietnamese reports follow VAS and SBV circulars (e.g. Circular 31/2024 on loan classification, Circular 41/2016 and 22/2019 on capital and liquidity ratios); every field mapping, unit and year-to-date convention must be audited in the same automated way, and state-owned banks may need separate treatment.
- **Point-in-time and governance:** supervisory data are confidential, so the pipeline would run on-premise with access control and must store every report vintage, which the SBV can do and the public FDIC API cannot.

References and tools

1. FDIC BankFind Suite API and field dictionaries, https://api.fdic.gov/banks/docs/ (accessed November 2026).
2. BFT65B course materials: exam paper and starter script fdic_download_starter.py (adapted in src/01_download.py).
3. FDIC (1998). Managing the Crisis: The FDIC and RTC Experience 1980–1994.
4. Cole, R. A. & White, L. J. (2012). Déjà Vu All Over Again: The Causes of U.S. Commercial Bank Failures This Time Around. Journal of Financial Services Research 42, 5–29.
5. Polars (v1.44), pandas (v3.1), PyArrow, PySpark (v4.3), scikit-learn (v1.9) documentation.
6. AI assistance: Claude (Anthropic) via Claude Code was used to help write and debug the pipeline code, the report generator and to draft text. The author reviewed and is responsible for all design choices, code, results and text.

Appendix

A. Data-quality summary (generated by src/03_quality.py)

Table 10. All data-quality checks, findings and handling.

Check	Finding	Affected	Handling in the pipeline
Completeness	0 quarters with row count != API total	0	download re-run until 0 (resumable manifest)
Primary key	0 duplicate (CERT, REPDTE)	0	assert in panel build (pipeline fails if > 0)
Structural missingness	fields not collected in early years: OTHBFHLB, P3ASSET	2	never imputed with future info: missing-indicator + training-block median; columns empty in a training block are dropped for that block
Negative stock amounts	negative values in fields that cannot be negative	860	set to null (treated as a reporting error)
Extreme ratios	ratios far outside economic range (tiny denominators)	21397	ratios recomputed from amounts with safe division; winsorised at train-period 0.5/99.5 pct
Zero used as missing	risk-based capital ratios are 0 instead of null (pre-1990, and CBLR banks from 2020)	100% of 1984-89 rows	capital ratios not used as model features; equity/assets (eq_ta), which exists in all eras, is used instead
Negative equity	insolvent banks report EQ < 0	15659	kept: genuine (and highly informative) signal, not an error
Accounting identities	small share of rows break balance-sheet identities	12940	rows kept: DEP > ASSET coincides with negative equity (insolvency), the rest is <0.05% of rows; ratios use safe division and winsorising
YTD flows	income items are cumulative within the calendar year	all rows	de-cumulated: flow_q = YTD_q - YTD_{q-1} (same year, consecutive quarter), Q1 = YTD; annualised x4; null when previous quarter missing
Reporting gaps	missing quarters inside a bank's reporting history	32 banks	lags/changes computed on the quarter index, not on row order; null when gap
Failures without Call Reports	in-scope failure events whose CERT never appears in Financials	6	cannot be labelled; reported as a (tiny) coverage limitation
Reports after FAILDATE	failed institution keeps filing after FAILDATE (1989 RTC conservatorships)	271	rows with REPDTE >= FAILDATE are dropped; label uses the first FAILURE date
Repeated events per CERT	assistance followed by failure, or two failures	44	first RESTYPE = FAILURE event defines the label; assistance is not a failure
Exit classification	banks that stop reporting: failure vs. merger/other	19751 exited banks	failure iff CERT in Failures with RESTYPE=FAILURE; other exits keep label 0 and leave the panel after their last report
Orphan CERTs	CERTs in Financials missing from Institutions	235	kept (Institutions not needed for features)
Static attributes	Institutions holds current, not historical, BKCLASS/STNAME/SPECGRP	4518 banks	use quarter-specific BKCLASS/STNAME from Financials
Exit fields	ACTIVE/ENDEFYMD/CHANGEC are future information	all	never used as features
Schema drift	column dtypes differ between quarter files	20	explicit schema cast in 02_storage.py

B. Completeness by quarter (excerpt)

Table 11. Rows per quarter vs. API total (all 170 quarters match; full list in results/dq_rows_per_quarter.csv).

Quarter	Rows	Banks	API total	Match
1984-03-31	17,930	17,930	17,930	yes
1984-06-30	17,935	17,935	17,935	yes
1984-09-30	17,946	17,946	17,946	yes
1984-12-31	17,949	17,949	17,949	yes
1994-03-31	13,230	13,230	13,230	yes
2004-03-31	9,229	9,229	9,229	yes
2009-03-31	8,337	8,337	8,337	yes
2014-03-31	6,812	6,812	6,812	yes
2019-03-31	5,428	5,428	5,428	yes
2024-03-31	4,640	4,640	4,640	yes
2025-12-31	4,411	4,411	4,411	yes
2026-03-31	4,353	4,353	4,353	yes
2026-06-30	4,313	4,313	4,313	yes

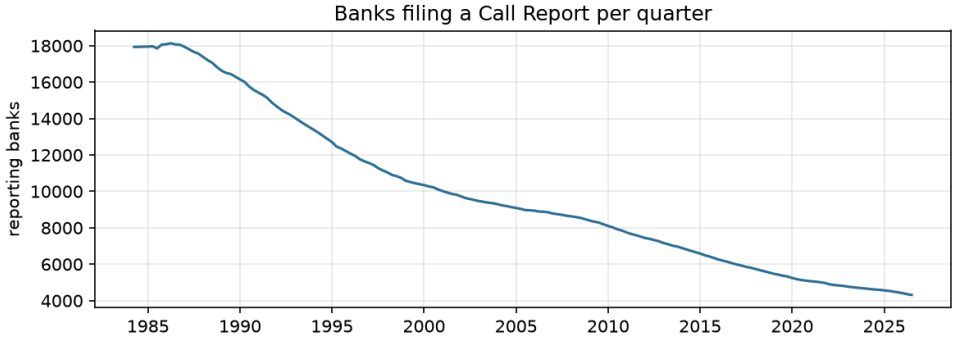


Figure A1. Banks filing a Call Report per quarter, 1984–2026.

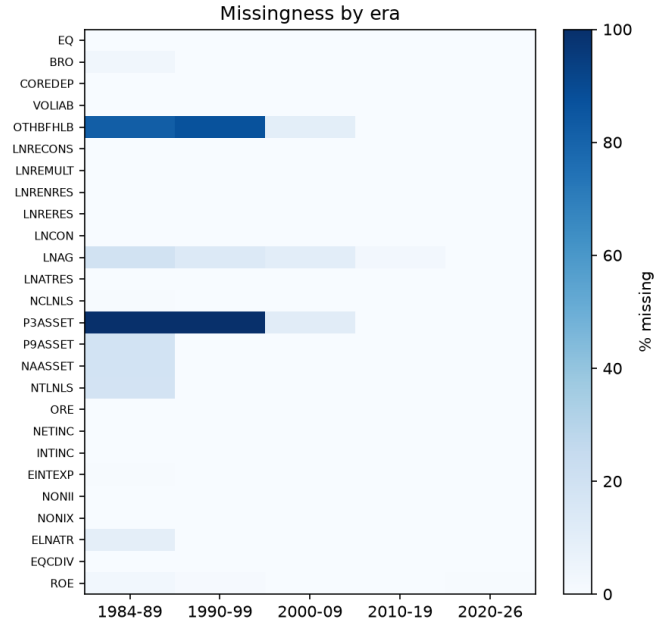


Figure A2. Share of missing values by field and era.

C. Feature dictionary

Table 12. Feature dictionary (labelled rows up to 2025Q2).

Feature	Group	Rationale	Missing %	Median, y=1	Median, y=0

eq_ta	C	equity cushion against losses	0.2	0.0167	0.0925
npl_ln	A	non-current loans share	1.3	0.0985	0.0092
npa_ta	A	non-performing assets incl. repossessed RE	0.1	0.1071	0.0072
pd30_ta	A	early delinquency, leads NPLs	58.3	0.0235	0.0041
ore_ta	A	foreclosed real estate	0.3	0.0305	0.0005
texas	A	Texas ratio: bad assets / loss-absorbing capital	0.4	1.666	0.0688
alll_ln	A	loan-loss reserve level	1.4	0.0306	0.0126
nco_ln	A	annualised net charge-offs	6.1	0.019	0.0005
prov_ta	A	annualised provisions	2.9	0.0105	0.0011
efficiency	M	cost / income	1.7	1.206	0.6683
roa_q	E	annualised quarterly ROA (de-cumulated)	0.6	-0.0291	0.0096
nim_q	E	annualised net interest margin	0.7	0.0222	0.0371
div_payer	E	pays dividends this quarter	0.0	0	0
liq_ta	L	liquid assets	0.0	0.1916	0.3057
loans_ta	L	loan-to-assets	0.0	0.6797	0.6132
coredep_ta	L	stable funding	0.1	0.8088	0.7736
brokered_dep	L	hot money	1.7	0	0
volliab_ta	L	volatile liabilities	0.2	0.1626	0.0978
fhlb_ta	L	wholesale borrowings	49.7	0.0599	0.0092
cof_ta	S	cost of funds (rate sensitivity)	0.7	0.0613	0.0309
cre_ta	S	commercial RE concentration	0.2	0.172	0.1068
constr_ta	S	construction & land development	0.2	0.0298	0.0135
resid_ln	S	1-4 family mortgages	1.3	0.2558	0.272
ci_ln	S	C&I loans	1.1	0.1157	0.1268
cons_ln	S	consumer loans	1.4	0.0718	0.0903
ag_ln	S	agricultural loans	12.7	0	0.0141
log_assets	size	size (too-big-to-fail, diversification)	0.0	11.23	11.35
d4_eq_ta	dyn	change over 4 quarters (deterioration speed)	5.9	-0.0309	0.0007
d4_npa_ta	dyn	change over 4 quarters (deterioration speed)	5.8	0.0286	-0.0001
d4_npl_ln	dyn	change over 4 quarters (deterioration speed)	6.9	0.025	-0.0001
d4_roa_q	dyn	change over 4 quarters (deterioration speed)	6.3	-0.015	0.0001
g4_asset	dyn	4-quarter growth (rapid growth precedes failures)	5.7	-0.0827	0.0526
g4_lnlsgr	dyn	4-quarter growth (rapid growth precedes failures)	6.7	-0.1106	0.063
roa_vol8	dyn	earnings volatility, trailing 8 quarters	4.9	0.0384	0.0039
roa_4q	dyn	trailing 4-quarter mean ROA	2.0	-0.0346	0.0095
n_reports	dyn	length of reporting history (de novo risk)	0.0	21	46
state_npa_med	ctx	state-level stress, same quarter	0.0	0.0228	0.0071
state_roa_med	ctx	state-level stress, same quarter	0.0	0.0059	0.0098
npa_vs_state	ctx	bank NPA relative to state median	0.1	0.0815	-0
fails_trailing4q_us	ctx	US failures in the past 4 quarters (public)	0.0	201	25

D. Walk-forward results for all models and folds

Table 13. Per-fold walk-forward results.

Model	Fold	Val. positives	PR-AUC	ROC-AUC	Recall top 5%
Baseline: logistic, 5 CAMELS ratios	train<= 1987-12 | val 1989-03..1990-12	2736	0.243	0.823	50.6%
Baseline: logistic, 5 CAMELS ratios	train<= 1989-12 | val 1991-03..1992-12	1157	0.214	0.939	79.0%
Baseline: logistic, 5 CAMELS ratios	train<= 1991-12 | val 1993-03..1995-12	153	0.097	0.974	95.4%
Baseline: logistic, 5 CAMELS ratios	train<= 1994-12 | val 1996-03..2006-12	168	0.099	0.877	72.0%
Logistic, all 40 features (L2)	train<= 1987-12 | val 1989-03..1990-12	2736	0.327	0.779	49.2%
Logistic, all 40 features (L2)	train<= 1989-12 | val 1991-03..1992-12	1157	0.375	0.965	82.9%
Logistic, all 40 features (L2)	train<= 1991-12 | val 1993-03..1995-12	153	0.135	0.972	94.8%
Logistic, all 40 features (L2)	train<= 1994-12 | val 1996-03..2006-12	168	0.133	0.888	69.6%
Histogram gradient boosting (HGB)	train<= 1987-12 | val 1989-03..1990-12	2736	0.458	0.948	68.2%
Histogram gradient boosting (HGB)	train<= 1989-12 | val 1991-03..1992-12	1157	0.516	0.977	87.3%
Histogram gradient boosting (HGB)	train<= 1991-12 | val 1993-03..1995-12	153	0.378	0.992	97.4%
Histogram gradient boosting (HGB)	train<= 1994-12 | val 1996-03..2006-12	168	0.136	0.920	76.2%

E. Optimised Polars plan for the panel (excerpt)

```
 WITH_COLUMNS:
 [((col("fail_qi") > col("qi")) & (col("fail_qi") <= (col("qi") + 4)))).fill_null([false]).strict_cast(Int8).alias("y"),
(col("fail_qi") - col("qi")).alias(...
  FILTER (col("FAILDATE").is_null() | (col("REPDTE") < col("FAILDATE")))
  FROM
    LEFT JOIN:
    LEFT PLAN ON: [col("CERT")]
      simple π 57/57 ["CERT", "REPDTE", "NAME", ... 54 other columns]
         WITH_COLUMNS:
         [when(col("__POLARS_CSER_0x100000042")).then(col("NETINC")).otherwise(when(col("qi").shift([dyn int: 1]).over([col("CERT")])
== col("__POLARS_CSER_0...
           WITH_COLUMNS:
           [(col("REPDTE").dt.month() == 3).alias("__POLARS_CSER_0x100000042"), (col("qi") - 1).alias("__POLARS_CSER_0x100000044")]
            SORT BY [col("CERT"), col("qi")]
               WITH_COLUMNS:
               [when(col("ASSET") < 0).then(null.cast(Int64)).otherwise(col("ASSET")).alias("ASSET"), when(col("DEP") <
0).then(null.cast(Int64)).otherwise(c...
                Parquet SCAN [C:/Users/ASUS/Desktop/Môn Bigdata/Final exam/data/curated/financials/year=1984/00000000.parquet, ... 42
other sources]
                PROJECT 48/49 COLUMNS
                SELECTION: col("ASSET") > 0
                ESTIMATED ROWS: 1711016
    RIGHT PLAN ON: [col("CERT")]
       WITH_COLUMNS:
       [((col("FAILDATE").dt.year() * 4) + ((col("FAILDATE").dt.month().cast(Int32) - 1) // 3)).alias("fail_qi")]
        AGGREGATE[maintain_order: false]
          [col("FAILDATE").min()] BY [col("CERT")]
          FROM
          simple π 2/2 ["CERT", "FAILDATE"]
            Parquet SCAN [C:/Users/ASUS/Desktop/Môn Bigdata/Final exam/data/curated/failures.parquet]
            PROJECT 3/16 COLUMNS
            SELECTION: (col("CERT").is_not_null() & col("RESTYPE").str.to_uppercase().is_in([["FAILURE"]]))
...
```

F. Reproducibility

python -m venv .venv; pip install -r requirements.txt; python run_all.py. The download is resumable and verified, all seeds are fixed (42), data files are not committed, and this report is regenerated with python report/build_report.py. Full instructions: README.md.